

Technical Documentation: Hybrid CNN-ResNet50 Deep Learning Architecture for Image Classification

System Documentation

July 8, 2026

Contents

1	Executive Summary	2
2	System Preprocessing & Data Pipeline	2
2.1	Automated Corrupted Image Scrubbing	2
2.2	Input Pipeline Loading Parameters	2
3	Hybrid Network Architecture	3
3.1	Conceptual Overview	3
3.2	Mathematical Representation	3
3.2.1	Branch A: The Scratch Convolutional Network	3
3.2.2	Branch B: ResNet50 Transfer Learning Backbone	3
3.2.3	Concatenation and Classification Top	3
4	Loss Function and Optimization	3
5	Model Implementation Code	4

1 Executive Summary

This document details a Deep Learning software workflow optimized for robust computer vision pipelines. The system targets data pipeline reliability via an automated corrupted-image parsing sweep, followed by feature extraction through a unified **Hybrid Feature Fusion Model**. By fusing local representations from a tailored Convolutional Neural Network (CNN) with macro-level abstractions from a deep pre-trained ResNet50 backbone, the system maximizes structural performance while minimizing overfitting through spatial data augmentations.

2 System Preprocessing & Data Pipeline

2.1 Automated Corrupted Image Scrubbing

Deep learning execution loops often crash due to truncated assets or incomplete data transfers (e.g., zero-byte file writes, metadata truncation). To prevent stream compilation errors, an aggressive automated verification step scans target directories recursively.

Files are discarded if they fail either of two programmatic tests:

1. **Allocation Validation:** Checks if file size evaluates to exactly 0 KB.
2. **Bitstream Decoding:** Attempts structural extraction using Python’s Pillow library via the `verify()` method to trap structural corruption.

2.2 Input Pipeline Loading Parameters

Images are processed in batches using highly optimized asynchronous pipelines. The static runtime hyper-parameters are documented below:

Table 1: Data Pipeline and Parameter Specifications		
Hyper-parameter	Target Value	Technical Justification
Image Resolution	48×48 pixels	Balanced profile matching low-res facial datasets.
Batch Size	64	Empirically optimal for parallel tensor streams.
Learning Rate	0.0005	Prevents gradient destruction in frozen layers.
Interpolation Mode	'nearest'	Ensures high edge-contrast retention during downscaling.

The dataset tensors are cast to float values and normalized globally over the feature space to map absolute grayscale/RGB intensities from standard intervals:

$$x_{\text{norm}} = \frac{x}{255.0}$$

Performance bottlenecks are mitigated by mapping inputs asynchronously using standard caching mechanisms (`.cache()`) and thread pre-fetching pipelines configured dynamically via `tf.data.AUTOTUNE`.

3 Hybrid Network Architecture

3.1 Conceptual Overview

Instead of relying solely on linear, single-lane networks, this system implements a **Parallel Feature Fusion Hybrid Network**. Utilizing the Keras Functional API, a shared input tensor branches simultaneously into two asymmetric processing paths before convergence.

3.2 Mathematical Representation

Let the input space be a tensor denoted as $X \in \mathbb{R}^{H \times W \times C}$. The shared data augmentation mapping transforms X into an augmented variant:

$$X_{\text{aug}} = \mathcal{A}(X)$$

3.2.1 Branch A: The Scratch Convolutional Network

The custom CNN captures micro-level visual properties (such as distinct edges, grain patterns, and localized textures). It applies sequential convolutions and downsampling steps:

$$f_{\text{CNN}}(X_{\text{aug}}) = \text{Flatten}\left(\text{MaxPool}\left(\sigma(\text{Conv2D}(X_{\text{aug}}))\right)\right)$$

3.2.2 Branch B: ResNet50 Transfer Learning Backbone

Concurrently, a deep residual network model initialized with pre-trained ImageNet parameters extracts robust contextual shapes:

$$f_{\text{ResNet}}(X_{\text{aug}}) = \text{GlobalAveragePooling2D}\left(\mathcal{W}_{\text{ResNet50}}(X_{\text{aug}})\right)$$

Where model weights $\mathcal{W}$ remain completely static (`trainable = False`) to preserve feature foundational structures.

3.2.3 Concatenation and Classification Top

The output matrices are flattened, mapped into a single dimensional domain, and structurally concatenated together side-by-side:

$$Z = f_{\text{CNN}}(X_{\text{aug}}) \oplus f_{\text{ResNet}}(X_{\text{aug}})$$

The final state mapping evaluates to class conditional probability distribution vector $\hat{y}$ via a Softmax output dense layer:

$$\hat{y} = \text{Softmax}\left(W_2 \cdot \max(0, W_1 Z + b_1) + b_2\right)$$

4 Loss Function and Optimization

The optimization goal minimizes prediction error calculated across individual integer class allocations using categorical cross-entropy loss metrics:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{i,j} \log(\hat{y}_{i,j})$$

Adaptive Moment Estimation (Adam) regulates updating sequences using customized learning parameter targets to smooth high-dimensional gradients.

5 Model Implementation Code

The architectural logic responsible for constructing this hybrid network is configured via the Python code block below:

```
1 # Shared Input Layer
2 input_layer = layers.Input(shape=(IMG_HEIGHT, IMG_WIDTH, 3))
3
4 # Shared Data Augmentation Layer
5 augmented = layers.RandomFlip("horizontal")(input_layer)
6 augmented = layers.RandomRotation(0.1)(augmented)
7
8 # --- Branch 1: Custom CNN ---
9 cnn_branch = layers.Conv2D(32, (3, 3), activation='relu')(augmented)
10 cnn_branch = layers.MaxPooling2D((2, 2))(cnn_branch)
11 cnn_branch = layers.Conv2D(64, (3, 3), activation='relu')(cnn_branch)
12 cnn_branch = layers.MaxPooling2D((2, 2))(cnn_branch)
13 cnn_branch = layers.Conv2D(128, (3, 3), activation='relu')(cnn_branch)
14 cnn_branch = layers.MaxPooling2D((2, 2))(cnn_branch)
15 cnn_branch = layers.Flatten()(cnn_branch)
16
17 # --- Branch 2: ResNet50 ---
18 base_model = ResNet50(include_top=False, weights='imagenet', input_shape=(
19     IMG_HEIGHT, IMG_WIDTH, 3))
20 base_model.trainable = False
21 resnet_branch = base_model(augmented)
22 resnet_branch = layers.GlobalAveragePooling2D()(resnet_branch)
23
24 # --- Merge Branches ---
25 merged = layers.concatenate([cnn_branch, resnet_branch])
26
27 # --- Classification Top ---
28 x = layers.Dense(256, activation='relu')(merged)
29 x = layers.Dropout(0.5)(x)
30 output_layer = layers.Dense(len(class_names), activation='softmax')(x)
31
32 # Create Hybrid Model
33 hybrid_model = models.Model(inputs=input_layer, outputs=output_layer)
```

Listing 1: Keras Functional API Code Block for Hybrid Fusion